UET NOWSHERA | ELECTRICAL ENGINEERING

LAB REPORT 12

POINTERS IN C++

COURSE: Computer Programming
AUTHOR: M.Younas Khan
ROLL NO: [25NWELE0741]

1. OBJECTIVE

- To understand how computer memory and variables interact.
- To define and utilize pointers in C++.
- To learn pointer declaration and memory address allocation.
- To apply the referencing (&) and dereferencing (*) operators.

2. THEORETICAL BACKGROUND

Computer Memory and Variables. The computer's memory is made up of bytes, each with a specific address. A variable in a program is assigned a specific block of memory to hold its value. For example, a float variable typically requires 4 bytes of contiguous memory to store its binary representation.

Pointers. In C++, a pointer is a variable that points to or references a memory location in which data is stored. Put another way, the pointer does not hold a value in the traditional sense; instead, it holds the address of another variable. Because a pointer holds an address, it connects directly to the underlying memory architecture.

Referencing Operator. To store the address of a variable in a pointer, we use the unary '&' (address-of) operator. For instance, ptr = &k; retrieves the memory address of the variable k and copies it into the pointer variable ptr.

Dereferencing Operator. The dereferencing operator is the asterisk (*). It is used to access or modify the value stored at the memory address the pointer is holding. Writing *ptr = 7; will copy the value 7 to the memory address pointed to by ptr, effectively altering the original variable.

3. SOFTWARE USED

- IDE: DEV-C++ IDE
- Compiler: C++ Compiler (bundled with DEV-C++)

4. EXPERIMENTAL TASKS

Task 1: Pointer Declaration and Initialization

An integer variable was declared and initialized. A pointer variable was then declared and assigned the memory address of the integer variable using the referencing operator. Both the direct value of the variable and the value accessed via the dereferencing operator were displayed to verify they matched.

```
#include <iostream>
using namespace std;

int main() {
    // Declare and initialize an integer variable
    int num = 42;

    // Declare a pointer and initialize it to the address of num
    int *ptr = &num;

    // Display values
    cout << "Value of num directly: " << num << endl;
    cout << "Address of num (&num): " << &num << endl;
    cout << "Address stored in ptr: " << ptr << endl;
    cout << "Value pointed to by ptr (*ptr): " << *ptr << endl;

    return 0;
}
```

Task 2: Pointer Arithmetic

An integer array was declared and populated. A pointer was initialized to point to the first element of the array. A loop utilizing pointer arithmetic was then used to iterate through the contiguous memory locations of the array, displaying each element's value.

```
#include <iostream>
using namespace std;

int main() {
    // Declare an integer array
    int numbers[] = {10, 20, 30, 40, 50};

    // Declare a pointer and initialize it to the first element
    int *ptr = numbers; // equivalent to &numbers[0]

    // Use pointer arithmetic to access elements
    cout << "Array elements using pointer arithmetic:" << endl;
    for (int i = 0; i < 5; i++) {
```

```
        cout << "Element " << i << ": " << *(ptr + i) << endl;
    }

    return 0;
}
```

5. CONCLUSION

This lab introduced the powerful concept of pointers in C++. By directly accessing and manipulating memory addresses using the referencing and dereferencing operators, programs can achieve greater efficiency and finer control over data storage. The application of pointer arithmetic to traverse arrays highlighted how contiguous memory blocks are handled at a low level. Understanding pointers is essential for advanced programming tasks involving dynamic memory allocation and complex data structures.